

Christian Santiago

Software & Cloud Engineer – Distributed Systems & GenAI Infrastructure on AWS
Agentic Retrieval Systems (RAG), Document Extraction, Data Pipelines, Evaluation, Guardrails
System Design, Event-Driven Architecture, Infrastructure as Code

santiagothedeveloper@gmail.com | christiansantiago.dev | linkedin.com/in/christian-santiago-dev | github.com/Go-Santiago-Go

SUMMARY

Software engineer building generative AI infrastructure on AWS in Go, Python, TypeScript, and Terraform. Four years of infrastructure monitoring and incident response through an on-premises-to-hybrid AWS migration, then ETL pipelines, dimensional modeling, and business software integration delivered directly to clients and non-technical stakeholders.

Currently focused on agentic RAG and document extraction in healthcare as part of Pfizer's AI document insights program. It builds on three systems I built on Amazon Bedrock and Amazon SageMaker: a retrieval system that grounds and cites every answer; an inference gateway with rate limiting, cost metering, and provider failover; and an MLOps retraining pipeline that automates data governance in CI.

EXPERIENCE

Pfizer / Extern

AI Document Insights & Data Extraction Extern

Aug. 2026 – Present

- Building an AI document extraction prototype replacing manual review of non-GMP clinical supply documents: scanned files parsed in Python with PyMuPDF, OpenCV, and OCR (Tesseract, PaddleOCR) to extract key fields and classify documents.
- Implementing retrieval over the extracted corpus with Retrieval Augmented Generation (RAG) pipelines, metadata filtering, LlamaIndex, LangGraph, and a vector database.
- Deploying a searchable interface for querying the extracted document corpus.

Amazon Web Services Cloud Institute

Cloud Engineering Resident

Sep. 2025 – Jul. 2026

- Built event-driven workflows with AWS Lambda, Amazon API Gateway, Amazon DynamoDB, Amazon S3, Amazon SNS, and Amazon SQS, orchestrating multi-step processes with AWS Step Functions, and deploying containerized applications with Docker, Amazon ECS, Amazon EKS, and Amazon ECR.
- Built LLM applications on Amazon Bedrock with LangChain, chaining model calls and managing conversation memory, grounded through RAG over Amazon Bedrock Knowledge Bases, with prompt engineering and Amazon Bedrock Guardrails.
- Applied Amazon Rekognition and Amazon Textract for image analysis and document extraction, with least-privilege IAM and observability through Amazon CloudWatch and AWS X-Ray.
- Provisioned infrastructure as code in CloudFormation, AWS CDK, and Terraform, and built CI/CD pipelines with AWS CodeBuild and CodePipeline.

Price Printing

Business Systems Analyst

May 2025 – Feb. 2026

- Built and maintained ETL pipelines in Python standardizing client-provided mailing address data to USPS formatting standards and verifying deliverability before print, for batches of up to 250,000 addresses where client-typical 8–15% defect rates meant reprints, wasted postage, and cascading delays.
- Modeled reporting data dimensionally with star and snowflake schemas, isolating analytical queries from transactional traffic, surfaced through SQL and BI dashboards.
- Integrated CRM, order management, and inventory systems, resolving the cross-platform data inconsistencies that were breaking fulfillment workflows.
- Presented operational findings to leadership and worked with non-technical stakeholders to turn reporting into decisions.

Freelance

Freelance Developer

Aug. 2022 – Aug. 2026

- Delivered short-cycle contract engagements for clients integrating their systems with industry-specific CRMs and third-party APIs.
- Engineered automated ETL workflows to ingest, normalize, and validate raw client data from disparate third-party sources, enforcing the schema consistency required for trusted downstream workflows.
- Built serverless applications on AWS for client workloads in React and Node.js.

UPS

Infrastructure & Monitoring Analyst

Sep. 2018 – Aug. 2022

- Contributed to an on-premise to hybrid AWS migration, working hands-on with Lambda, API Gateway, S3, RDS, DynamoDB, SNS/SQS, and CloudWatch across backend infrastructure supporting facility operations.

- Developed incident response and monitoring for an 80,000-package sort, one of four daily sorts, setting CloudWatch thresholds against a 15–20 minute downtime target where faults drove missorts and service failures.
- Transformed post-mortem incident reports into dimensional OLAP models, surfacing the recurring faults behind extended downtime so they could be addressed before repeating.
- Decoupled facility systems with SNS and SQS event-driven workflows to route operational alerts across the distribution environment.

PROJECTS

rag-api

Retrieval

- Retrieval Augmented Generation (RAG) service in Go: chunking, Amazon Bedrock embeddings, pgvector search, and cross-encoder reranking to the five passages that build the prompt and return as its citations, with generation constrained to them so an unanswerable question gets a refusal rather than an invention. Retrieval quality is measured rather than assumed: an offline harness over 35 hand-labelled questions took passage recall@5 from 57.1% to 77.1%, with zero hallucinated claims across 70 graded responses. ECS Fargate, Terraform, GitHub OIDC.

inference-gateway

Serving

- A raw model endpoint has no per-caller identity, no throttling, no failover, and no answer to “what is p95 right now,” so every team calling it rebuilds the same retry loop and nobody can attribute the bill. This is that layer, in Go: SSE token streaming cancellable mid-stream, per-key auth and token-bucket rate limiting decided in middleware so a rejected request never reaches a paid API, ordered multi-provider routing behind per-backend circuit breakers, and every response priced by the model that actually answered. roughly 23 microseconds of serving overhead per request, 8.6 MB distroless image. Amazon Bedrock with an Ollama fallback, a React and TypeScript client, ECS Fargate and Terraform.

retrain-pipeline

Training and governance

- CI-driven MLOps pipeline making data quality a merge requirement and model promotion a human decision: labeled data enters by pull request as a DVC pointer, Great Expectations checks gate the merge with the failing one named in the report, and the dataset content hash becomes the model’s identity so a registry entry walks back to the exact rows behind it. A Go CLI submits SageMaker training and files candidates as PendingManualApproval; the CI role deliberately lacks UpdateModelPackage, so promotion is enforced by an absent permission rather than by policy.

TECHNICAL SKILLS

Languages & Web:	Go, Python, TypeScript, JavaScript, SQL, Bash, HTML/CSS, React, Node.js
GenAI & MLOps:	Bedrock (Knowledge Bases, Guardrails), RAG, Embeddings, Vector Databases, pgvector, Semantic Search, Chunking Strategies, Reranking, Retrieval Evaluation, Document Extraction (Textract, OCR), LangChain, LangGraph, LangSmith, LlamaIndex, Prompt & Context Engineering, LLM Evaluation (Golden Datasets, LLM-as-Judge, Hallucination Rate), Agentic Architectures, AWS Strands SDK, Bedrock AgentCore, Model Context Protocol (MCP), SageMaker (Training, Model Registry), scikit-learn, PyTorch
Data & Pipelines:	ETL/ELT, Apache Airflow, dbt, Databricks, PySpark, Dimensional Modeling (Star/Snowflake, OLAP), Data Quality Gates (Great Expectations), Dataset Versioning (DVC), pandas, NumPy, BI Dashboards
Infrastructure & Cloud (AWS):	Terraform (IaC), Docker, Linux, Lambda, API Gateway, ECS/Fargate, EKS, S3, DynamoDB, RDS/PostgreSQL, Step Functions, SNS, SQS, EventBridge, CloudFront, Route 53, VPC, IAM, Secrets Manager, ECR, CloudFormation, CDK, CodeBuild, CodePipeline
Reliability & Operations:	Incident Response, Postmortems, Runbooks, Architecture Decision Records, Observability (Metrics, Logs, Traces), Prometheus, Grafana, CloudWatch, X-Ray, Alerting & Thresholds, Circuit Breakers, Retries with Backoff, Multi-Provider Failover, FinOps/Cost Optimization
System Design:	Distributed Systems, Event-Driven Architecture, Serverless, Microservices, REST APIs, Least-Privilege IAM, Idempotency, Concurrency, Interface Boundaries & Dependency Inversion, CI/CD (GitHub Actions, GitHub OIDC)
Certifications:	AWS Certified Developer – Associate; AWS Certified Cloud Practitioner; AI Engineer for Data Scientists Associate (DataCamp); Data Engineer Associate (DataCamp); Data Science with AI Professional Certification (Codecademy)

EDUCATION

Amazon Web Services Cloud Institute – Cloud Application Developer	Remote, Sep. 2025 – Jul. 2026
Codesmith – Software Engineering + AI/ML Immersive	Remote, May 2021 – Apr. 2022
Vanderbilt University – Computer Science	Nashville, TN, Aug. 2014 – May 2017